MC458 - Projeto e Análise de Algoritmos I

Prof. Dr. Hilder Vitor Lima Pereira

Instituto de computação

UNICAMP

12 de março de 2026

1 Revisão

2 Algoritmos recursivos e resolução de recorrências
- Busca binária
- Resolução de recorrências por iteração e substituição
- Substituindo um candidato para resolvendo recorrências
- Método da árvore de recorrência

3 Perguntas ou comentários?

Revisão

Nas últimas aulas...

- algoritmos iterativos
  - estimar complexidade temporal
  - invariantes, subinvariantes e corretude de algoritmos
- algoritmos recursivos
  - conexão entre projeto e indução
  - expressar $T(n)$ usando recorrências
  - começamos a analisar complexidade de $T(n)$
- Notação assintótica: $O(f(n))$, $\Omega(f(n))$, $\omega(f(n))$, etc

Algoritmos recursivos e resolução de recorrências

Algoritmos recursivos e resolução de recorrências:

Busca binária

Busca num vetor

Problema

Entrada:

- vetor $v \in \mathbb{Z}^n$ e um inteiro $x \in \mathbb{Z}$

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Busca num vetor

Problema

Entrada:

- vetor $v \in \mathbb{Z}^n$ e um inteiro $x \in \mathbb{Z}$

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Sabemos que qualquer algoritmo para esse problema precisa, no mínimo, comparar x com todos os n elementos de v, tendo portanto complexidade temporal $\Omega(n)$.

Busca num vetor

Problema

Entrada:

- vetor $v \in \mathbb{Z}^n$ e um inteiro $x \in \mathbb{Z}$

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Sabemos que qualquer algoritmo para esse problema precisa, no mínimo, comparar x com todos os n elementos de v, tendo portanto complexidade temporal $\Omega(n)$.

No entanto, é possível buscar mais rapidamente se assumirmos que o vetor está ordenado.

Busca num vetor ordenado

Problema

Entrada:

- um inteiro $x \in \mathbb{Z}$ e um vetor $v \in \mathbb{Z}^n$ ordenado (crescente)

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Busca num vetor ordenado

Problema

Entrada:

- um inteiro $x \in \mathbb{Z}$ e um vetor $v \in \mathbb{Z}^n$ ordenado (crescente)

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?

Busca num vetor ordenado

Problema

Entrada:

- um inteiro $x \in \mathbb{Z}$ e um vetor $v \in \mathbb{Z}^n$ ordenado (crescente)

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?
´

- sabemos buscar em $v[0...n-2]$

Busca num vetor ordenado

Problema

Entrada:

- um inteiro $x \in \mathbb{Z}$ e um vetor $v \in \mathbb{Z}^n$ ordenado (crescente)

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?
´

- sabemos buscar em $v[0...n-2]$
- sabemos buscar em $v[0...n/2]$

Busca num vetor ordenado

Problema

Entrada:

- um inteiro $x \in \mathbb{Z}$ e um vetor $v \in \mathbb{Z}^n$ ordenado (crescente)

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?
´

- sabemos buscar em $v[0...n-2]$
- sabemos buscar em $v[0...n/2]$

Hipótese de indução

Para um inteiro $n \geq 1$, sabemos resolver o problema da busca em vetor ordenado para qualquer $x \in \mathbb{Z}$ e qualquer $v \in \mathbb{Z}^m$ com $m < n$.

Busca num vetor ordenado

Problema

Entrada:

- um inteiro $x \in \mathbb{Z}$ e um vetor $v \in \mathbb{Z}^n$ ordenado (crescente)

Saída:

- $i \in [\![0, n-1]\!]$ tal que $v[i] = x$ ou -1 se tal i não existir

Como projetar o algoritmo indutivamente? Qual subproblema utilizar?
´

- sabemos buscar em $v[0...n-2]$
- sabemos buscar em $v[0...n/2]$

Hipótese de indução

Para um inteiro $n \geq 1$, sabemos resolver o problema da busca em vetor ordenado para qualquer $x \in \mathbb{Z}$ e qualquer $v \in \mathbb{Z}^m$ com $m < n$.

Que algoritmo conseguimos projetar assim?

Busca binária

```
BuscaBinaria(v)
Input: x ∈ ℤ e v ∈ ℤ^n tal que v está ordenado (crescente)
if n = 0 then
    return −1
i_m = ⌊n/2⌋
if x = v[i_m] then
    return i_m
if x < v[i_m] then
    return BuscaBinaria(v[0, ..., i_m − 1])
else
    return BuscaBinaria(v[i_m + 1, ..., n − 1])
```

Complexidade temporal

Note que os valores da entrada não crescem arbitrariamente na execução do algoritmo, então, podemos estimar a complexidade em termos de operações elementares.

Primeira tentativa

$$T(n) = \begin{cases} 2, & n = 0 \\ 4, & n > 0 \text{ e } x = v[i_m], \\ T(\lfloor n/2 \rfloor) + k', & n > 0 \text{ e } x < v[i_m], \\ T(n - \lfloor n/2 \rfloor - 1) + k'', & n > 0 \text{ e } x > v[i_m] \end{cases}$$

Complexidade temporal

Note que os valores da entrada não crescem arbitrariamente na execução do algoritmo, então, podemos estimar a complexidade em termos de operações elementares.

Primeira tentativa

$$T(n) = \begin{cases} 2, & n = 0 \\ 4, & n > 0 \text{ e } x = v[i_m], \\ T(\lfloor n/2 \rfloor) + k', & n > 0 \text{ e } x < v[i_m], \\ T(n - \lfloor n/2 \rfloor - 1) + k'', & n > 0 \text{ e } x > v[i_m] \end{cases}$$

Nos concentrando em um limitante superior, podemos simplificar

$$T(n) \leq \begin{cases} k_1, & n = 0 \\ T(\lfloor n/2 \rfloor) + k_2, & n > 0 \end{cases}$$

Complexidade temporal

Note que os valores da entrada não crescem arbitrariamente na execução do algoritmo, então, podemos estimar a complexidade em termos de operações elementares.

Primeira tentativa

$$T(n) = \begin{cases} 2, & n = 0 \\ 4, & n > 0 \text{ e } x = v[i_m], \\ T(\lfloor n/2 \rfloor) + k', & n > 0 \text{ e } x < v[i_m], \\ T(n - \lfloor n/2 \rfloor - 1) + k'', & n > 0 \text{ e } x > v[i_m] \end{cases}$$

Nos concentrando em um limitante superior, podemos simplificar

$$T(n) \leq \begin{cases} k_1, & n = 0 \\ T(\lfloor n/2 \rfloor) + k_2, & n > 0 \end{cases}$$

Como encontrar a classe de complexidade de $T(n)$?

Algoritmos recursivos e resolução de recorrências:

Resolução de recorrências por iteração e substituição

Método da iteração e substituição

Vamos começar com a seguinte função, onde c é uma constante:

$$T(n) = \begin{cases} 0, & n = 0 \\ T(n-1) + c, & n > 0. \end{cases}$$

Método da iteração e substituição

Vamos começar com a seguinte função, onde c é uma constante:

$$T(n) = \begin{cases} 0, & n = 0 \\ T(n-1) + c, & n > 0. \end{cases}$$

Aplicamos a definição da função algumas vezes para tentar obter um candidato, um palpite, para o termo geral.

Método da iteração e substituição

Vamos começar com a seguinte função, onde c é uma constante:

$$T(n) = \begin{cases} 0, & n = 0 \\ T(n-1) + c, & n > 0. \end{cases}$$

Aplicamos a definição da função algumas vezes para tentar obter um candidato, um palpite, para o termo geral.
Temos

$$T(n) = T(n-k) + k \cdot c$$

Método da iteração e substituição

Vamos começar com a seguinte função, onde c é uma constante:

$$T(n) = \begin{cases} 0, & n = 0 \\ T(n-1) + c, & n > 0. \end{cases}$$

Aplicamos a definição da função algumas vezes para tentar obter um candidato, um palpite, para o termo geral.
Temos

$$T(n) = T(n-k) + k \cdot c$$

Usamos o caso base para obter uma expressão para $T(n)$:

$$n = k \Rightarrow T(n) = T(0) + n \cdot c = n \cdot c$$

Método da iteração e substituição

Vamos começar com a seguinte função, onde c é uma constante:

$$T(n) = \begin{cases} 0, & n = 0 \\ T(n-1) + c, & n > 0. \end{cases}$$

Aplicamos a definição da função algumas vezes para tentar obter um candidato, um palpite, para o termo geral.
Temos

$$T(n) = T(n-k) + k \cdot c$$

Usamos o caso base para obter uma expressão para $T(n)$:

$$n = k \Rightarrow T(n) = T(0) + n \cdot c = n \cdot c$$

Usamos indução matemática para provar que $T(n)$ realmente tem a fórmula fechada que encontramos

Decremento multiplicativo

Em vez de definirmos a relação de recorrência em função de um passo $n - k$, vamos considerar n/k.
Por exemplo, considerando apenas n como potência de 2:

$$f(n) = \begin{cases} 1 & \text{se } n \leq 1 \\ 2f(n/2) + n & \text{se } n > 1 \end{cases}$$

Decremento multiplicativo

Em vez de definirmos a relação de recorrência em função de um passo $n - k$, vamos considerar n/k.
Por exemplo, considerando apenas n como potência de 2:

$$f(n) = \begin{cases} 1 & \text{se } n \leq 1 \\ 2f(n/2) + n & \text{se } n > 1 \end{cases}$$

Aplicamos a definição da função algumas vezes para tentar obter um um palpite para o termo geral:

$$f(n) = 2^k \cdot f(n/2^k) + k \cdot n$$

Decremento multiplicativo

Em vez de definirmos a relação de recorrência em função de um passo $n - k$, vamos considerar n/k.
Por exemplo, considerando apenas n como potência de 2:

$$f(n) = \begin{cases} 1 & \text{se } n \leq 1 \\ 2f(n/2) + n & \text{se } n > 1 \end{cases}$$

Aplicamos a definição da função algumas vezes para tentar obter um um palpite para o termo geral:

$$f(n) = 2^k \cdot f(n/2^k) + k \cdot n$$

Usamos o caso base para obter uma expressão para $f(n)$:

$$k = \log n \Rightarrow f(n) = n \cdot f(1) + (\log n) \cdot n = n \cdot (1 + \log n)$$

Decremento multiplicativo

Em vez de definirmos a relação de recorrência em função de um passo $n - k$, vamos considerar n/k.
Por exemplo, considerando apenas n como potência de 2:

$$f(n) = \begin{cases} 1 & \text{se } n \leq 1 \\ 2f(n/2) + n & \text{se } n > 1 \end{cases}$$

Aplicamos a definição da função algumas vezes para tentar obter um um palpite para o termo geral:

$$f(n) = 2^k \cdot f(n/2^k) + k \cdot n$$

Usamos o caso base para obter uma expressão para $f(n)$:

$$k = \log n \Rightarrow f(n) = n \cdot f(1) + (\log n) \cdot n = n \cdot (1 + \log n)$$

Usamos indução matemática para provar que $f(n)$ realmente tem a fórmula fechada que encontramos.

Limitante superior

Nem sempre precisamos encontrar uma fórmula fechada. Muitas vezes, um limitante superior (ou inferior) já basta.
Voltando ao exemplo da busca binária, para simplificar a análise, limitamos a complexidade temporal como

$$T(n) \leq \begin{cases} c_1, & n = 0 \\ T(\lfloor n/2 \rfloor) + c_2, & n > 0 \end{cases}$$

Apliquemos o método da iteração e substituição...

Limitante superior

Nem sempre precisamos encontrar uma fórmula fechada. Muitas vezes, um limitante superior (ou inferior) já basta.
Voltando ao exemplo da busca binária, para simplificar a análise, limitamos a complexidade temporal como

$$T(n) \leq \begin{cases} c_1, & n = 0 \\ T(\lfloor n/2 \rfloor) + c_2, & n > 0 \end{cases}$$

Apliquemos o método da iteração e substituição...
Chegamos no palpite $T(n) \leq c \log n$ para alguma constante c e provamos por indução...

Resolvendo recorrências

Vimos o método da iteração e substituição:

- Iteramos algumas vezes para achar um padrão após k iterações
- Usamos o caso base para encontrar um candidato
- Usamos indução para provar que o candidato está certo

Resolvendo recorrências

Vimos o método da iteração e substituição:

- Iteramos algumas vezes para achar um padrão após k iterações
- Usamos o caso base para encontrar um candidato
- Usamos indução para provar que o candidato está certo

Exemplos:

- $T(n) = T(n-1) + c \Rightarrow T(n) \in O(n)$
- $T(n) = T(\lfloor n/2 \rfloor) + 1 \Rightarrow T(n) \in O(\log n)$
- $T(n) = 2T(\lfloor n/2 \rfloor) + n \Rightarrow T(n) \in O(n \log n)$

Algoritmos recursivos e resolução de recorrências:

Substituindo um candidato para resolvendo recorrências

Adivinhando uma solução

Considere a recorrência

$$T(n) = \begin{cases} 1 & \text{se } n = 17 \\ T(\lfloor (n-17)/2 \rfloor + 17) + 1 & \text{se } n \geq 18 \end{cases}$$

- Ela se parece muito com $\hat{T}(n) = \hat{T}(\lfloor n/2 \rfloor) + 1$, que já sabemos que é $O(\log n)$
- Mas parece mais difícil por causa do 17.

Adivinhando uma solução

Considere a recorrência

$$T(n) = \begin{cases} 1 & \text{se } n = 17 \\ T(\lfloor (n-17)/2 \rfloor + 17) + 1 & \text{se } n \geq 18 \end{cases}$$

- Ela se parece muito com $\hat{T}(n) = \hat{T}(\lfloor n/2 \rfloor) + 1$, que já sabemos que é $O(\log n)$
- Mas parece mais difícil por causa do 17.
- Intuitivamente, parece que $T(\lfloor n/2 \rfloor) \approx T(\lfloor (n-17)/2 \rfloor + 17)$.
- Então $T(n) \in O(\log n)$ também deve valer.
- Vamos demonstrar isso.

Cuidados com a notação assintótica

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2T(\lfloor n/2 \rfloor) + n & \text{se } n \geq 2, \end{cases}$$

- O uso descuidado da notação assintótica leva a erros
- Já sabemos que $T(n) = \Theta(n \log n)$
- Mas vamos "provar" que $T(n) = O(n)$!

Cuidados com a notação assintótica (cont)

Vamos mostrar que $T(n) \leq cn$ para alguma constante $c > 0$.

$$
\begin{aligned}
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\leq 2c \lfloor n/2 \rfloor + n \\
&\leq cn + n \\
&= O(n) \quad \longleftarrow \text{ERRADO!}
\end{aligned}
$$

Cuidados com a notação assintótica (cont)

Vamos mostrar que $T(n) \leq cn$ para alguma constante $c > 0$.

$$
\begin{aligned}
T(n) &= 2T(\lfloor n/2 \rfloor) + n \\
&\leq 2c \lfloor n/2 \rfloor + n \\
&\leq cn + n \\
&= O(n) \quad \longleftarrow \text{ERRADO!}
\end{aligned}
$$

Qual é o erro?

Cuidados com a notação assintótica (cont)

Vamos mostrar que $T(n) \leq cn$ para alguma constante $c > 0$.

$$\begin{aligned} T(n) &= 2T(\lfloor n/2 \rfloor) + n \\ &\leq 2c \lfloor n/2 \rfloor + n \\ &\leq cn + n \\ &= O(n) \quad \longleftarrow \text{ERRADO!} \end{aligned}$$

Qual é o erro?

- A nossa afirmação é que $T(n) \leq cn$
- Mas mostramos que $T(n) \leq (c+1)n$
- Não concluímos o passo indutivo

Algoritmos recursivos e resolução de recorrências:

Método da árvore de recorrência

Árvore de recorrência

Idéia:

1. crie uma árvore de recorrência:
   - os nós representam os termos independentes
   - os filhos representam as subfunções recorrentes
2. somamos os termos de cada nível da árvore
3. depois somamos todos os níveis

Vantagens

- útil quando há vários termos recorrentes
- é mais fácil organizar as contas

Exemplo

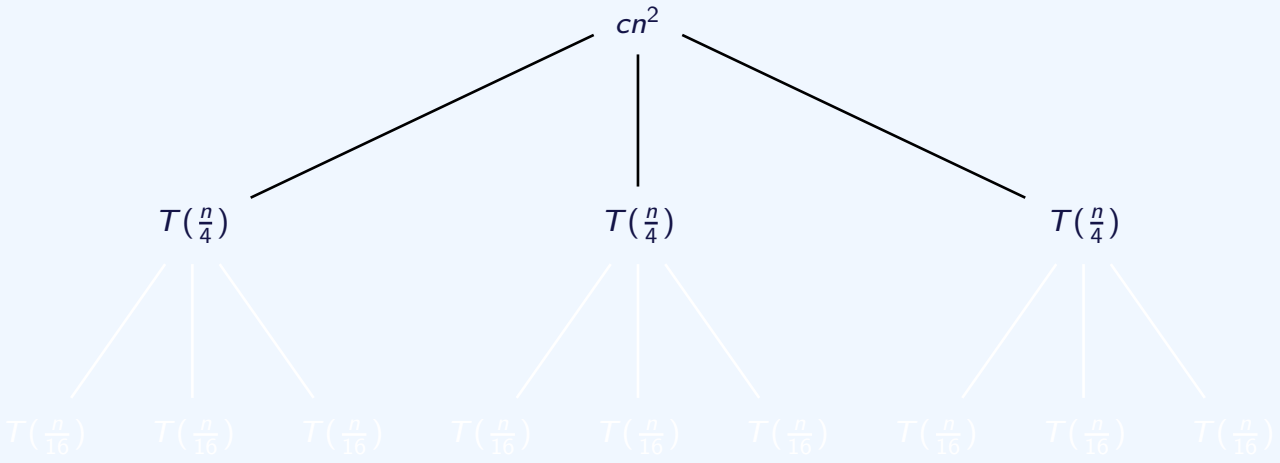
Encontre uma fórmula fechada para

$$T(n) = \begin{cases} 1 & \text{se } n \leq 3 \\ 3T(\lfloor n/4 \rfloor) + cn^2 & \text{se } n \geq 4 \end{cases}$$

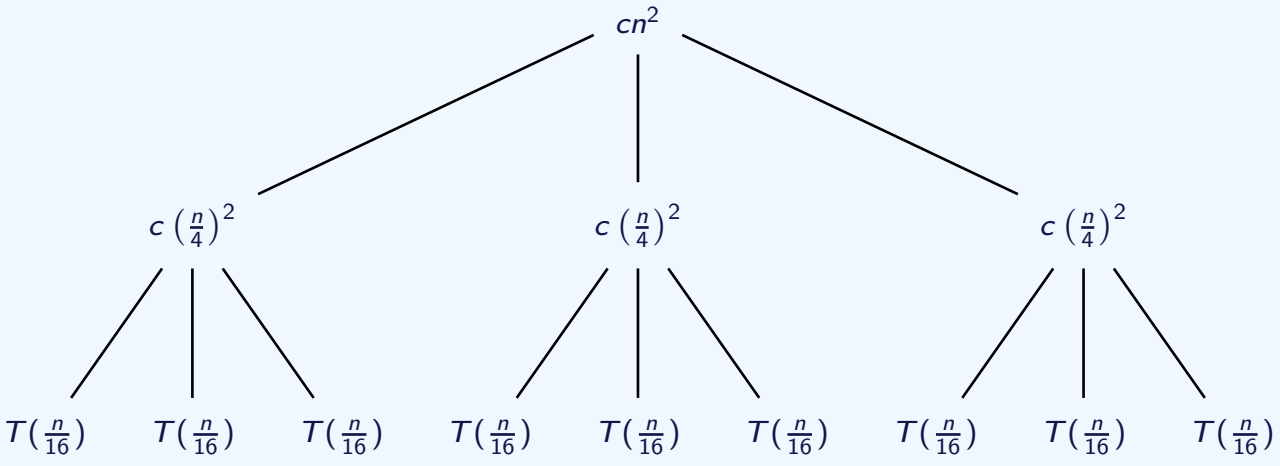
Simplificando

- Queremos obter uma fórmula fechada apenas
- De novo, supomos que $n = 4^k$
- Depois, verificamos com o método da substituição

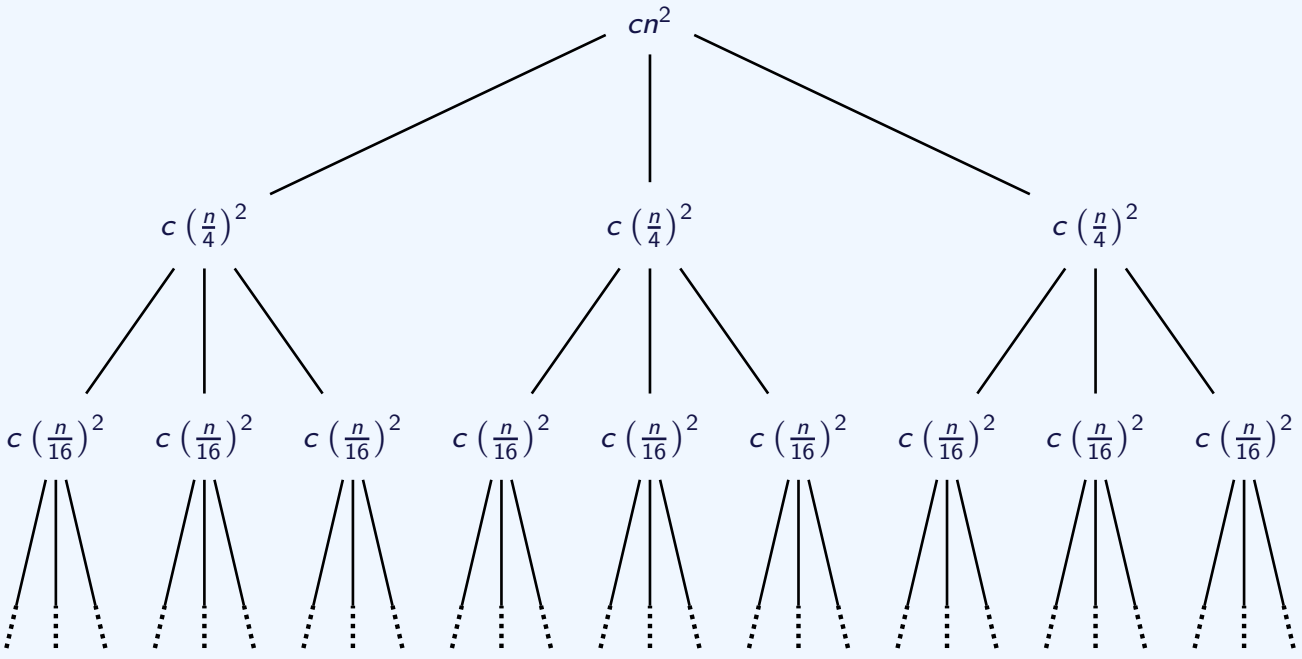
Árvore de recorrência



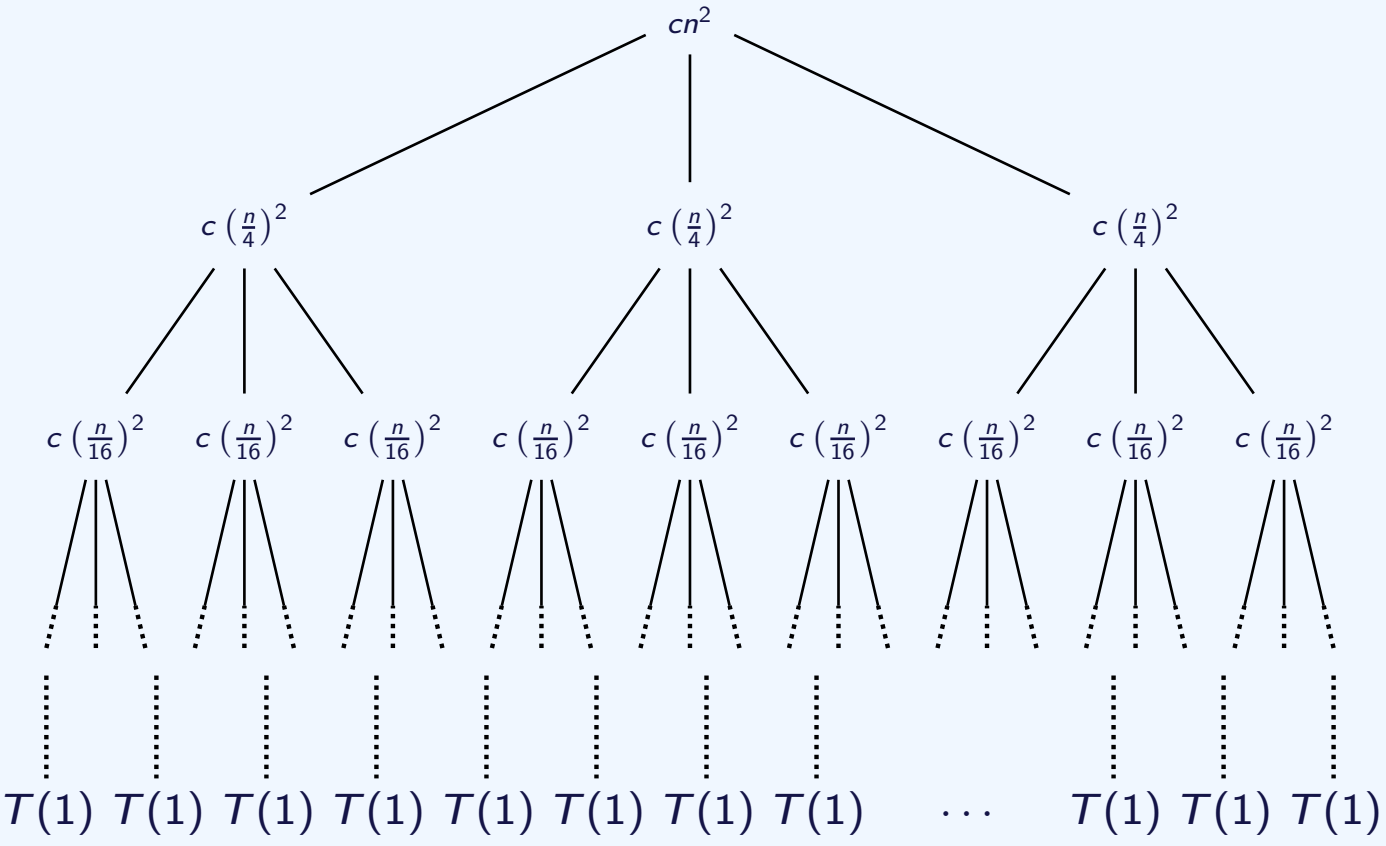
Árvore de recorrência



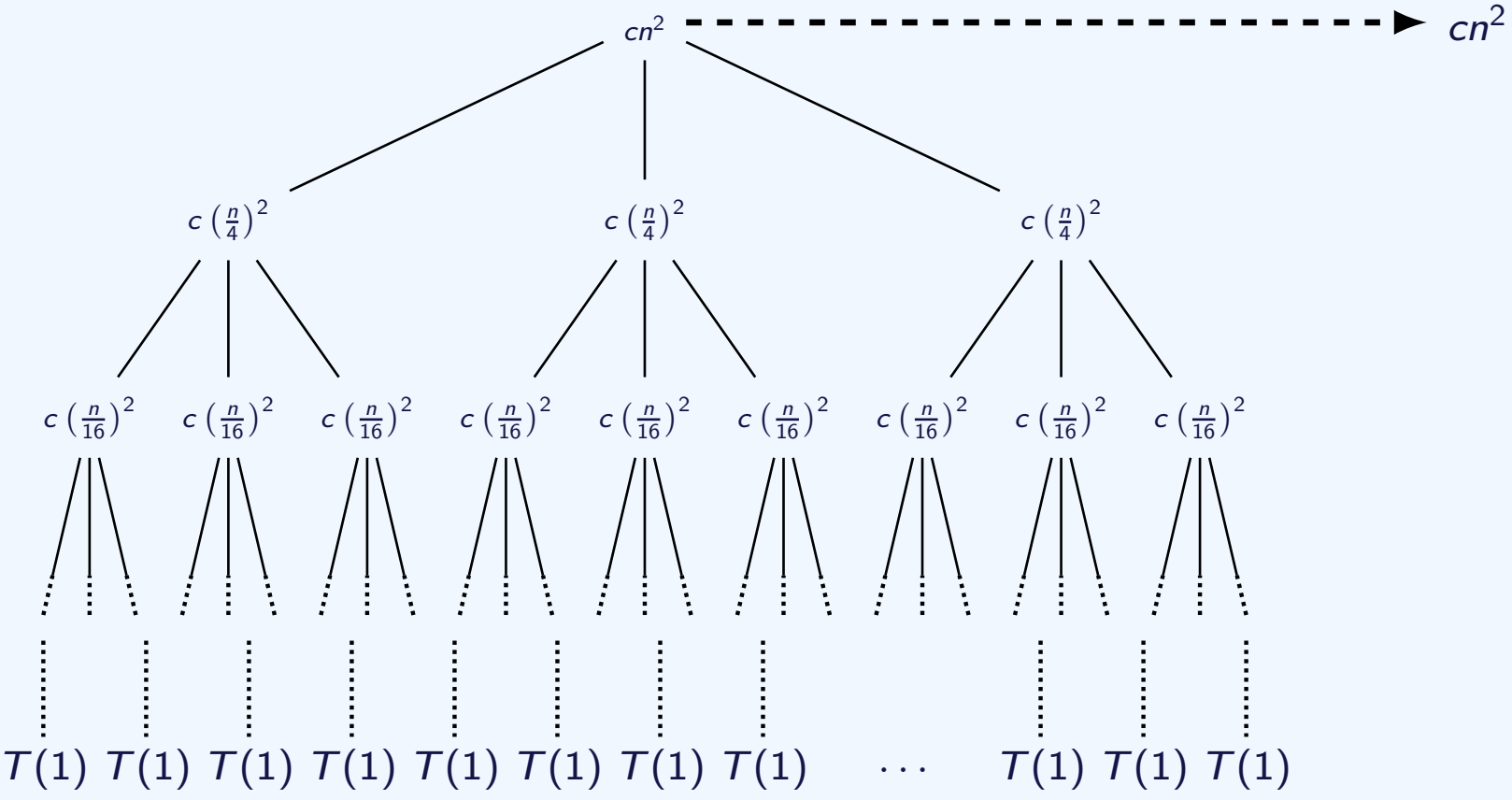
Árvore de recorrência



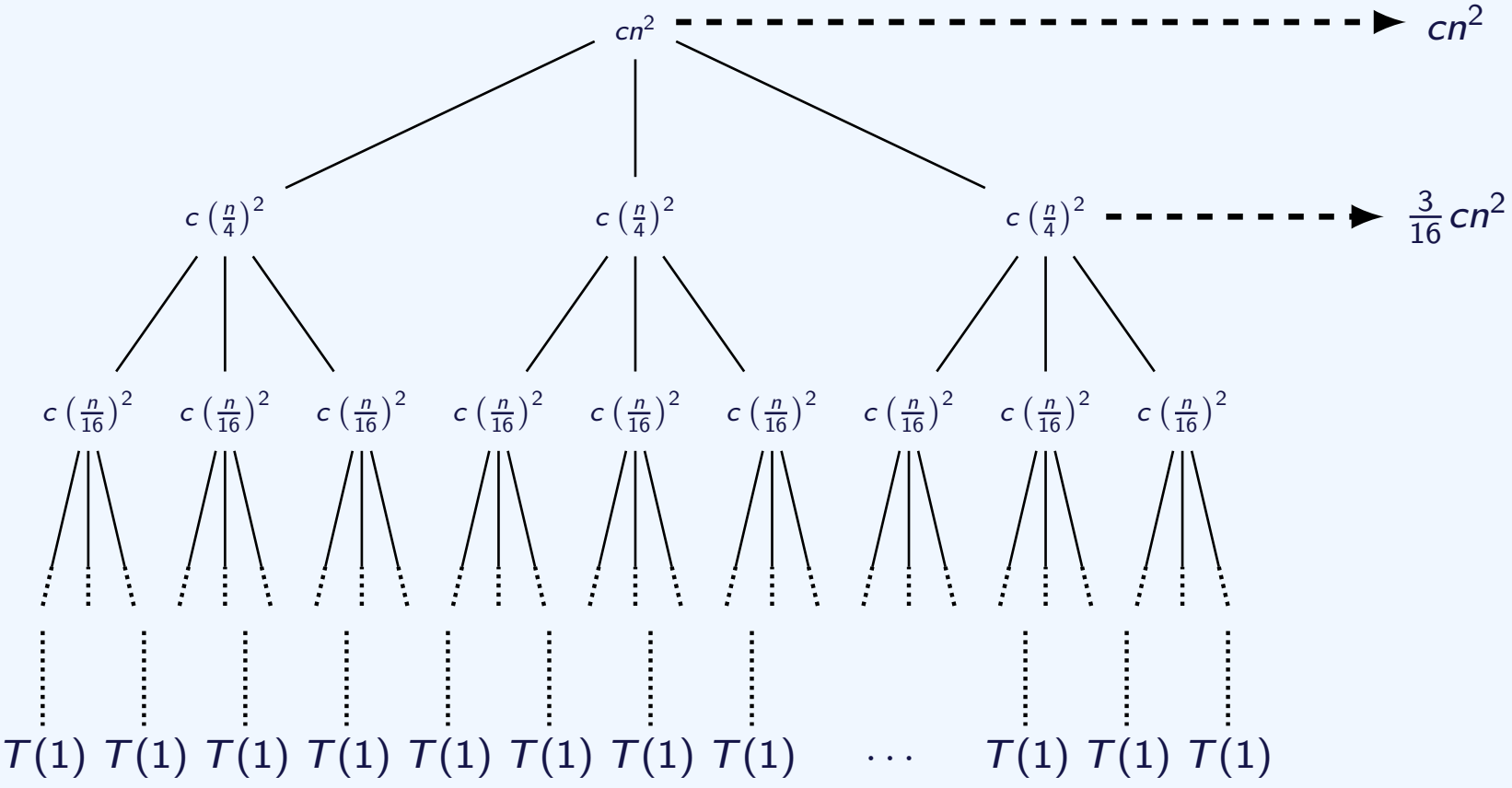
Árvore de recorrência



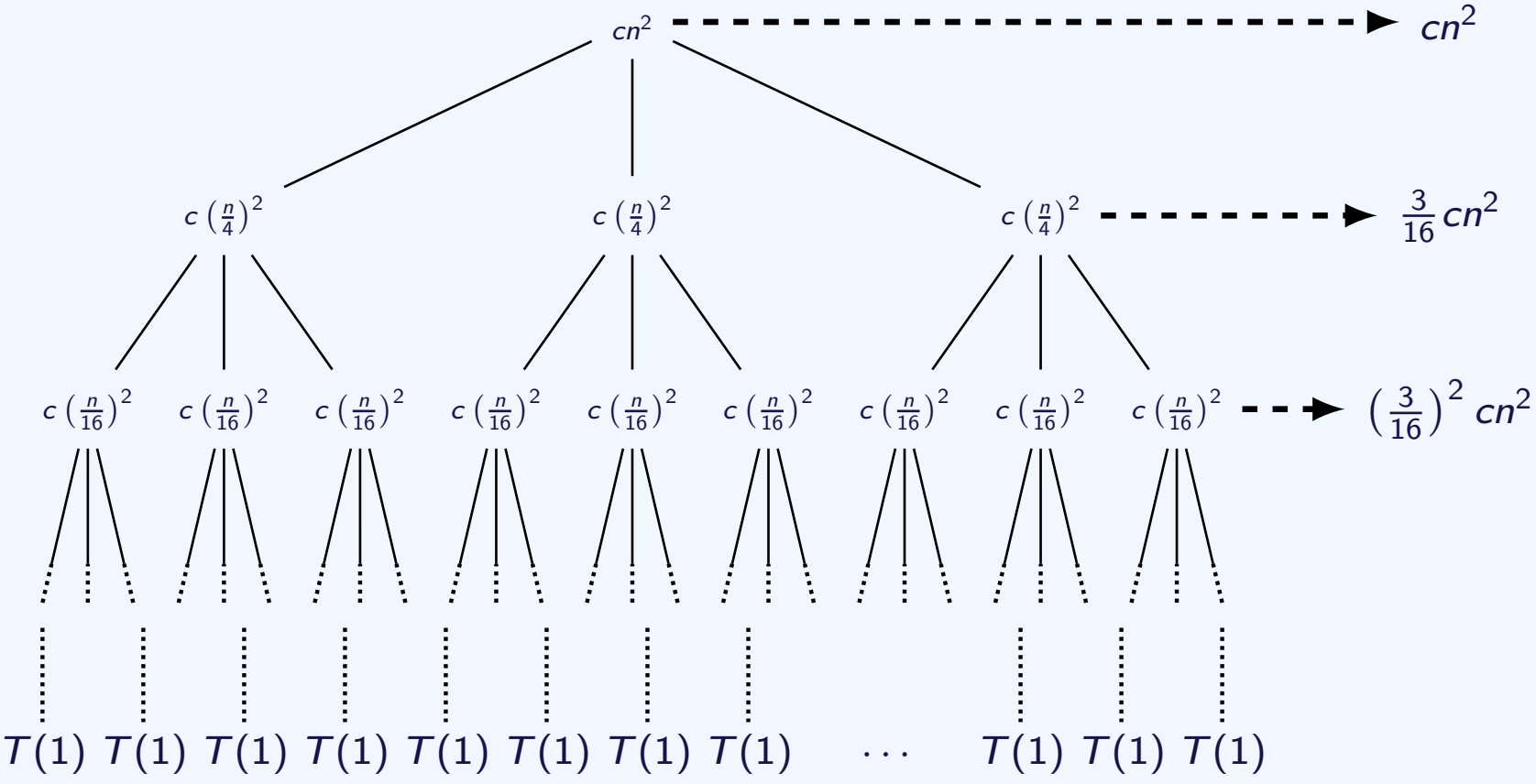
Árvore de recorrência



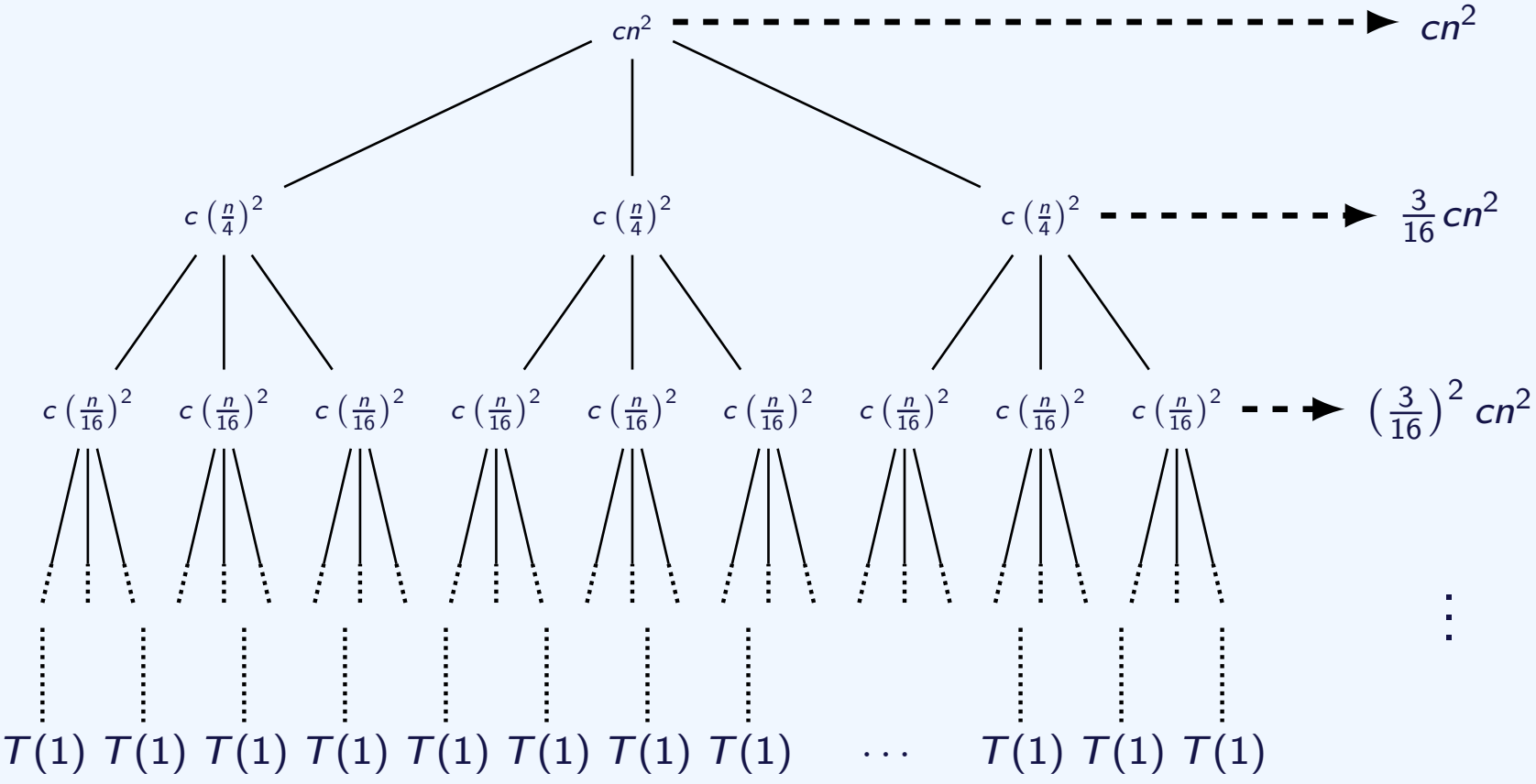
Árvore de recorrência



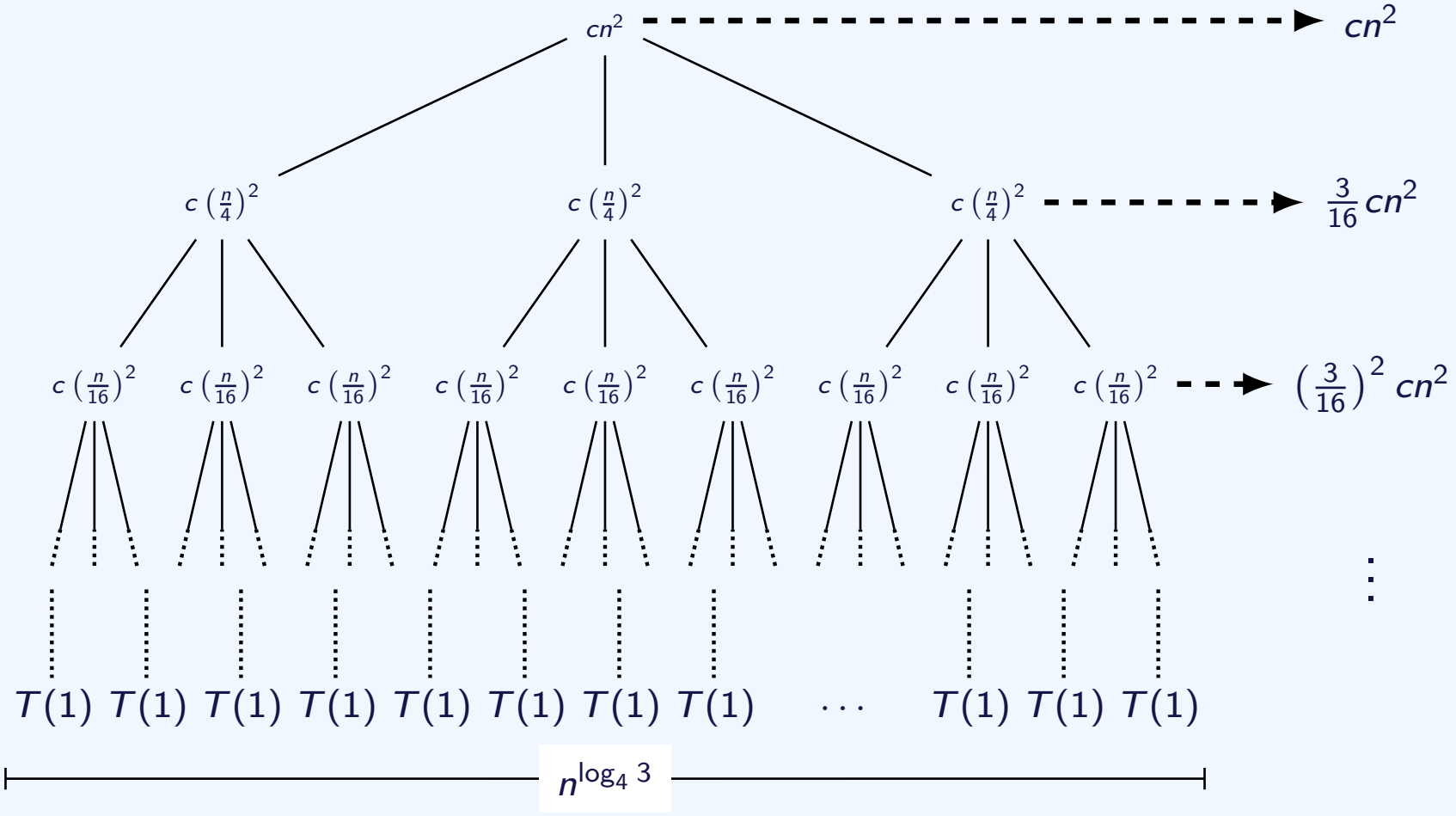
Árvore de recorrência



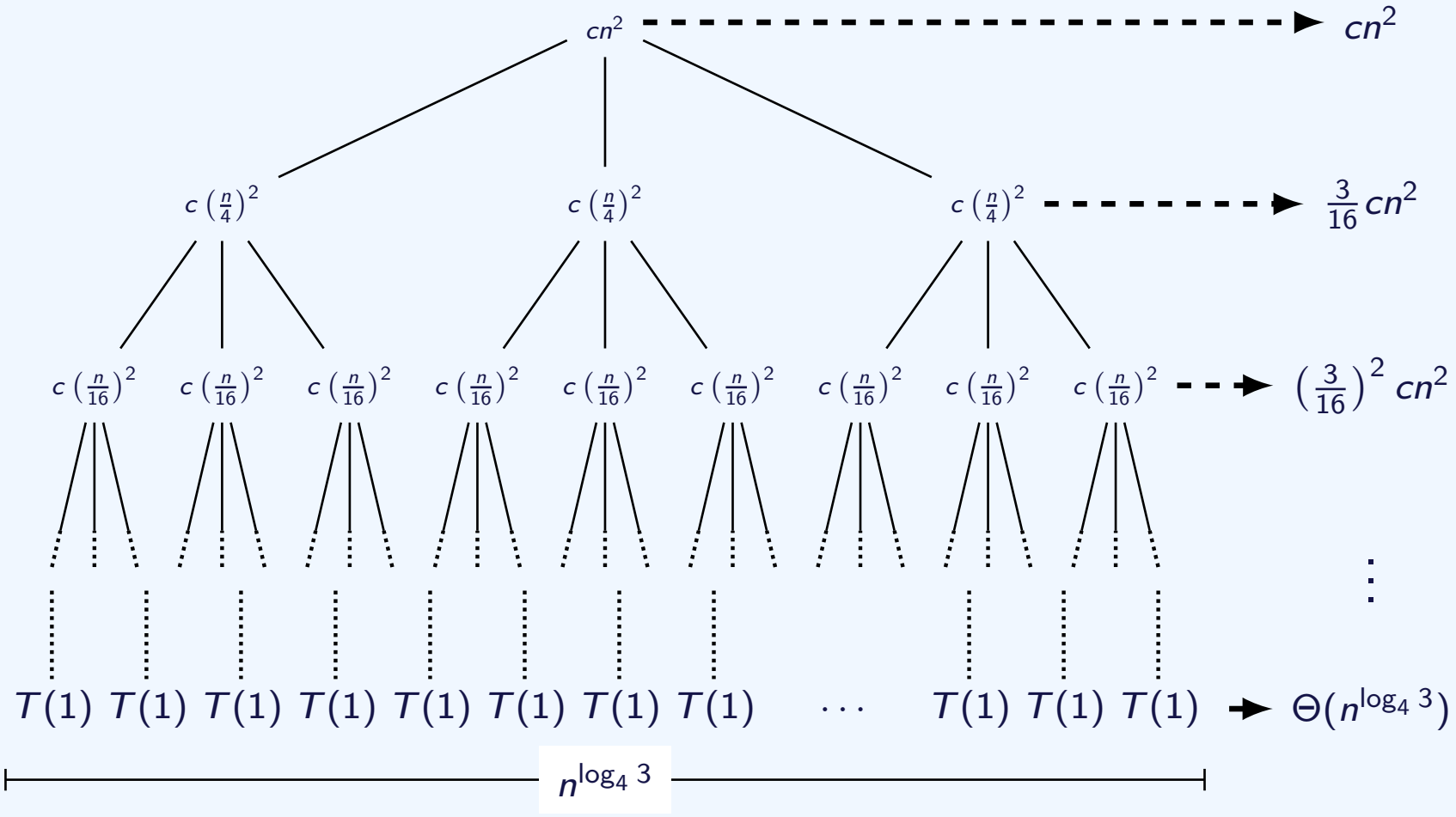
Árvore de recorrência



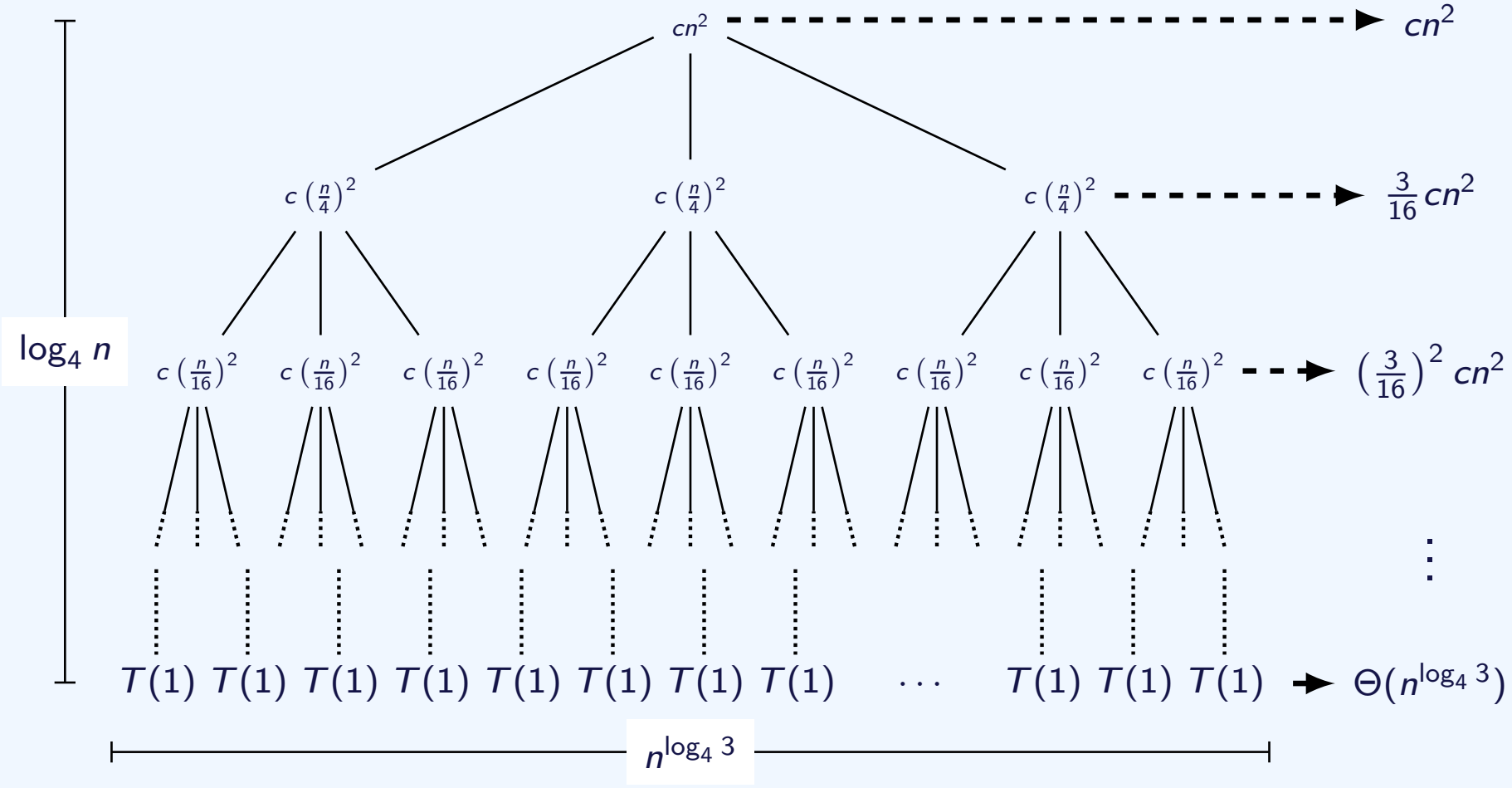
Árvore de recorrência



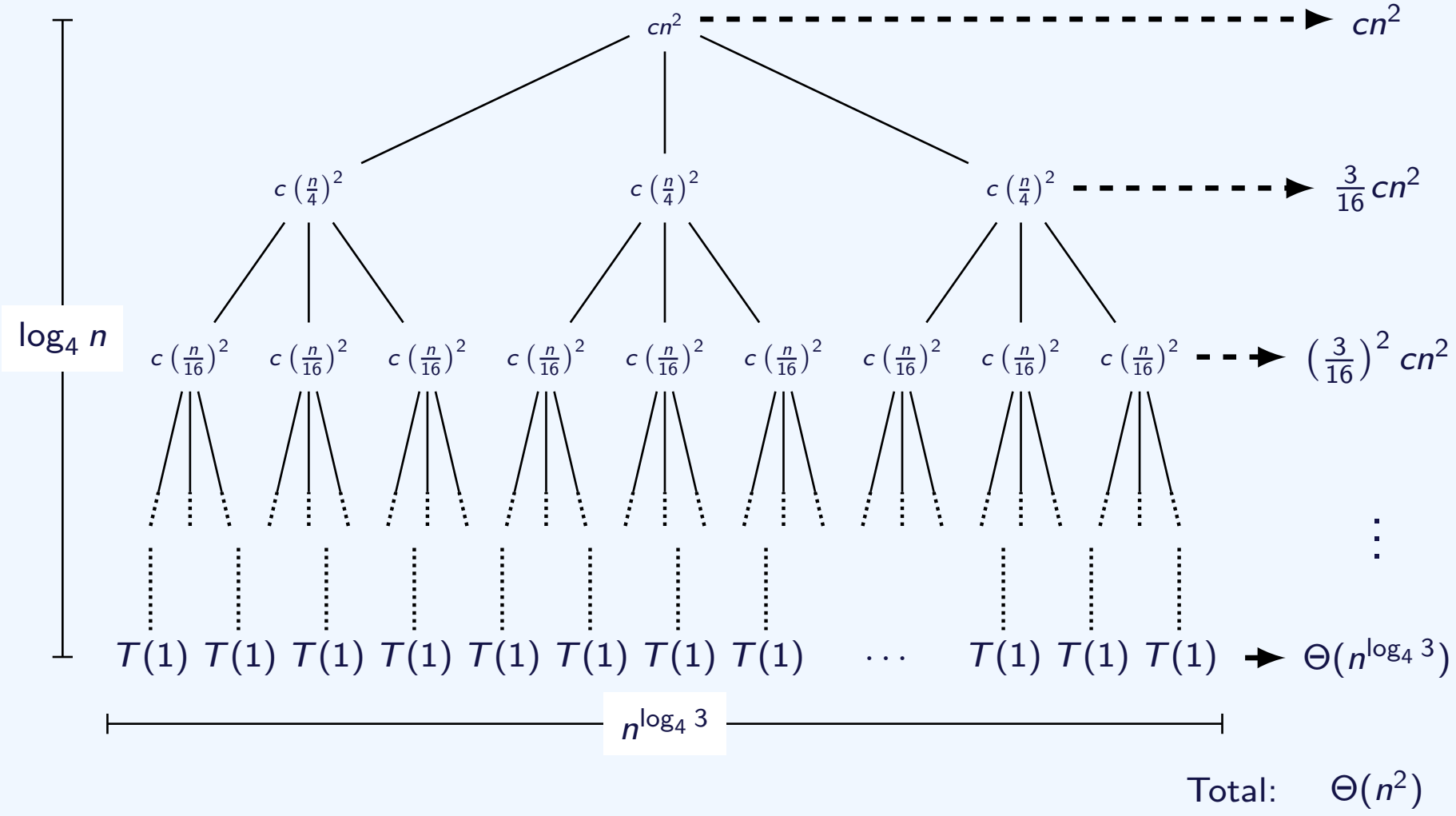
Árvore de recorrência



Árvore de recorrência



Árvore de recorrência



Total: $\Theta(n^2)$

Somando os termos

- A árvore tem altura $\log_4 n$

Somando os termos

- A árvore tem altura $\log_4 n$
- A soma de cada nível interno é $\left(\frac{3}{16}\right)^i cn^2$

Somando os termos

- A árvore tem altura $\log_4 n$
- A soma de cada nível interno é $\left(\frac{3}{16}\right)^i cn^2$
- O número de folhas é $3^{\log_4 n} = n^{\log_4 3}$

Somando os termos

- A árvore tem altura $\log_4 n$
- A soma de cada nível interno é $\left(\frac{3}{16}\right)^i cn^2$
- O número de folhas é $3^{\log_4 n} = n^{\log_4 3}$

$T(n)$

Somando os termos

- A árvore tem altura $\log_4 n$
- A soma de cada nível interno é $\left(\frac{3}{16}\right)^i cn^2$
- O número de folhas é $3^{\log_4 n} = n^{\log_4 3}$

$$T(n) = \left(cn^2 + \frac{3}{16} cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 \right) + \hat{c} \cdot n^{\log_4 3}$$

Somando os termos

- A árvore tem altura $\log_4 n$
- A soma de cada nível interno é $\left(\frac{3}{16}\right)^i cn^2$
- O número de folhas é $3^{\log_4 n} = n^{\log_4 3}$

$$
\begin{aligned}
T(n) &= \left(cn^2 + \frac{3}{16} cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 \right) + \hat{c} \cdot n^{\log_4 3} \\
&\leq \left(\sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i \right) cn^2 + \hat{c} \cdot n^{\log_4 3}
\end{aligned}
$$

Somando os termos

- A árvore tem altura $\log_4 n$
- A soma de cada nível interno é $\left(\frac{3}{16}\right)^i cn^2$
- O número de folhas é $3^{\log_4 n} = n^{\log_4 3}$

$$
\begin{aligned}
T(n) &= \left(cn^2 + \frac{3}{16} cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 \right) + \hat{c} \cdot n^{\log_4 3} \\
&\leq \left(\sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i \right) cn^2 + \hat{c} \cdot n^{\log_4 3} \\
&= \frac{16}{13} cn^2 + \hat{c} \cdot n^{\log_4 3},
\end{aligned}
$$

Somando os termos

- A árvore tem altura $\log_4 n$
- A soma de cada nível interno é $\left(\frac{3}{16}\right)^i cn^2$
- O número de folhas é $3^{\log_4 n} = n^{\log_4 3}$

$$
\begin{aligned}
T(n) &= \left(cn^2 + \frac{3}{16} cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 \right) + \hat{c} \cdot n^{\log_4 3} \\
&\leq \left(\sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i \right) cn^2 + \hat{c} \cdot n^{\log_4 3} \\
&= \frac{16}{13} cn^2 + \hat{c} \cdot n^{\log_4 3},
\end{aligned}
$$

Concluímos que $T(n) \in O(n^2)$.

Outro exemplo

Encontre uma fórmula fechada para

$$T(n) = \begin{cases} 1 & \text{se } n \leq 2 \\ T(\lceil n/3 \rceil) + T(\lfloor 2n/3 \rfloor) + n & \text{se } n \geq 3 \end{cases}$$

Esse exemplo é um pouco mais complicado

- Os termos recorrentes são diferentes
- A árvore não será completa
- Vamos mostrar juntos que $T(n) \in O(n \log n)$

Perguntas ou comentários?